PROJEKT:

REZEPT_EMPFEHLUNG

Phase2

Modulname: Projekt Software Engineering (DLMCSPSE01_D)
Name: Daniel Faller
Matrikelnummer: 10807727
Abgabedatum: 18.10.2026

Inhaltsverzeichnis

1.Projektübersicht & Zeitplan 2
2.Anforderungsdokument (User Stories) 3
3.Spezifikationsdokument & Datenmodell 4
4.Systemarchitektur & Schichtentrennung 5
4.1Technologieauswahl und Rationale 6
4.2 Architekturdokument: UML-Diagramme (Sequenz & Klassen) 6
4.3Prozessabläufe (Aktivitätsdiagramm) 7

1.Projektübersicht & Zeitplan

Das Ziel meines Projekts ist ein einfaches Python-Programm, das bei der täglichen Entscheidung hilft: "Was koche ich heute?" Das Programm fragt nach der Kategorie (Fleisch oder ohne Fleisch) und der verfügbaren Zeit (schnell oder mehr Zeit) und schlägt ein passendes Rezept vor.

Zeitplan:

- **Woche 1:** Idee aufschreiben, User Stories definieren und einfache UMLDiagramme zeichnen.
- **Woche 2-4:** Python-Code schreiben (Rezept und RezeptFinder Klassen) und die Oberfläche bauen.
- **Woche 5:** Das Programm testen, Fehler beheben und die Dokumentation fertigstellen.

Risikomanagement:

- **Risiko 1:** Probleme bei der Nutzung von Git und GitHub.
  - *Lösung:* Ich nutze nur die grundlegenden Befehle und lade die Dateien Schritt für Schritt hoch.
- **Risiko 2:** Schwierigkeiten beim Erstellen der UML-Diagramme.
  - *Lösung:* Ich zeichne die Diagramme mit dem kostenlosen Tool Draw.io.

Risiko	**Eintrittswahrscheinlichkeit**	**Schweregrad**	**Gegenmaßnahme / Lösung**
1. Versionskonflikte bei Git/GitHub	Mittel	Gering	eschränkung auf grundlegende GitBefehle (commit, push); schrittweises Hochladen.

Risiko	Eintrittswahrscheinlichkeit	Schweregrad	Gegenmaßnahme / Lösung
Schwierigkeiten bei UML-Diagrammen	Gering	Mittel	utzung des einfachen Online-Tools Draw.io für die Erstellung.
Logikfehler bei der Rezept-Filterung	Mittel	Mittel	anuelle Testläufe mit verschiedenen Kombinationen von Kategorien und Zeiten.

GitHub Repository

Der vollständige Quellcode und alle Projektdateien sind hier versioniert und abrufbar: ***https://github.com/fallerdani0808-dev/fallerdaniel.git***

2.Anforderungsdokument (User Stories)

- **Zielgruppe:** Personen, die schnell eine einfache Entscheidungshilfe für das Kochen suchen.
- **Entwickler:** Ein Einsteiger, der die Grundkonzepte der objektorientierten Programmierung anwendet.

User Stories:

- **US1:** Als Benutzer möchte ich auswählen können, ob ich Fleisch oder vegetarisch essen möchte.

- **US2:** Als Benutzer möchte ich angeben, ob es schnell gehen muss oder ob ich Zeit habe.
- **US3:** Als Benutzer möchte ich das Rezept sehen und am Ende entscheiden können, ob ich noch ein anderes Rezept suchen möchte.

Nicht-funktionale Anforderungen:

- Die Benutzeroberfläche soll übersichtlich und einfach zu bedienen sein.
- Das Programm soll stabil laufen und bei falschen Eingaben nicht einfach abstürzen.

Glossar:

- **GUI (Graphical User Interface):** Grafische Benutzeroberfläche zur Bedienung mit der Maus.
- **OOP (Objektorientierte Programmierung):** Strukturierung des Codes in Klassen und Objekte.
- **Tkinter:** Standard-Bibliothek in Python für grafische Fenster.

3.Spezifikationsdokument & Datenmodell

- **Datenmodell:** Es gibt eine Klasse Rezept für die einzelnen Gerichte (mit Namen, Kategorie, Zeit und Rezepttext) und eine Klasse RezeptFinder, welche die Rezepte speichert und filtert.
- **Geschäftsregel:** Ein Rezept wird nur vorgeschlagen, wenn Kategorie und Zeit genau zur Auswahl des Benutzers passen.
- **Benutzeroberfläche:** Ein einfaches Tkinter-Fenster mit Radiobuttons für die Auswahl und Klick-Buttons für die Steuerung

.

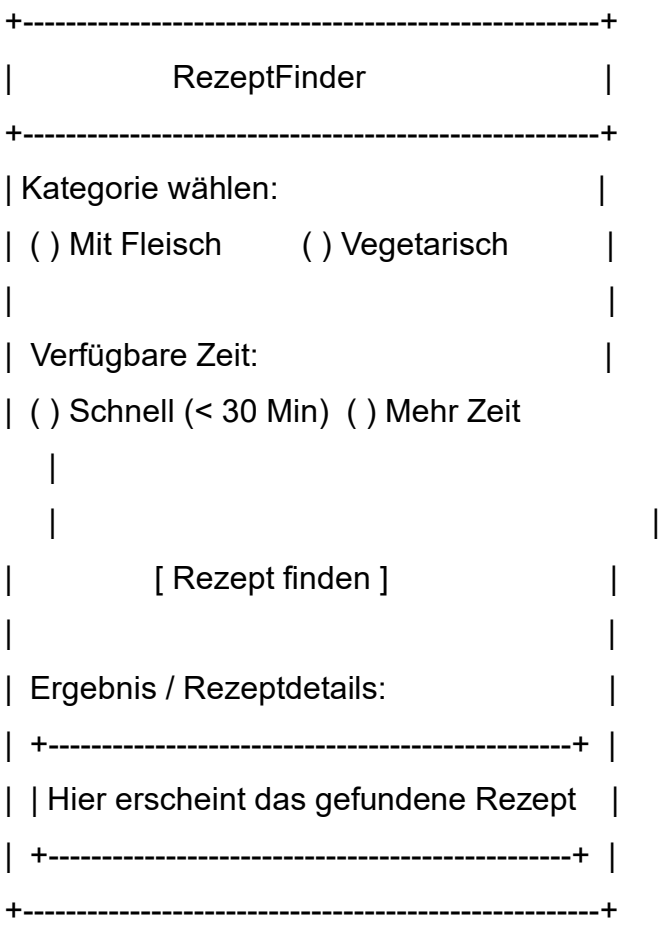
System- und Benutzerschnittstellen (Mock-up & Validierung)

- **Systemschnittstellen:** Da es sich um eine lokale Desktop-Anwendung handelt, gibt es keine komplexen Webschnittstellen. Die System-Schnittstellen beschränken sich auf die Python-internen Standardmodule (tkinter für die GUI, random für die Zufallsauswahl) sowie das lokale Betriebssystem.

- **Eingabevalidierung:** Wenn der Benutzer auf den Such-Button klickt, ohne eine Auswahl getroffen zu haben, fängt das Programm diesen Zustand im Code ab,

verhindert einen Absturz und gibt stattdessen einen Hinweis im Ausgabefenster aus.

- **Benutzerschnittstellen & Dialogfluss (Mock-up):** Das Programm besitzt eine klare, übersichtliche Dialogstruktur im Tkinter-Fenster:



4.Systemarchitektur & Schichtentrennung

Obwohl es sich um eine kleine Desktop-Anwendung handelt, folgt das Programm einer klaren **Logik- und Schichtentrennung** (angelehnt an ein einfaches Model-ViewControllerPrinzip), um den Code übersichtlich zu halten:

1. **Daten- und Modellschicht (Model):** Die Klasse Rezept verwaltet die reinen Daten eines Gerichts (Name, Kategorie, Zubereitungszeit und Text).

2. **Logikschicht (Controller/Logik):** Die Klasse RezeptFinder hält die Liste aller Rezepte und steuert die Filter- sowie Zufallsauswahl-Logik (random-Modul).

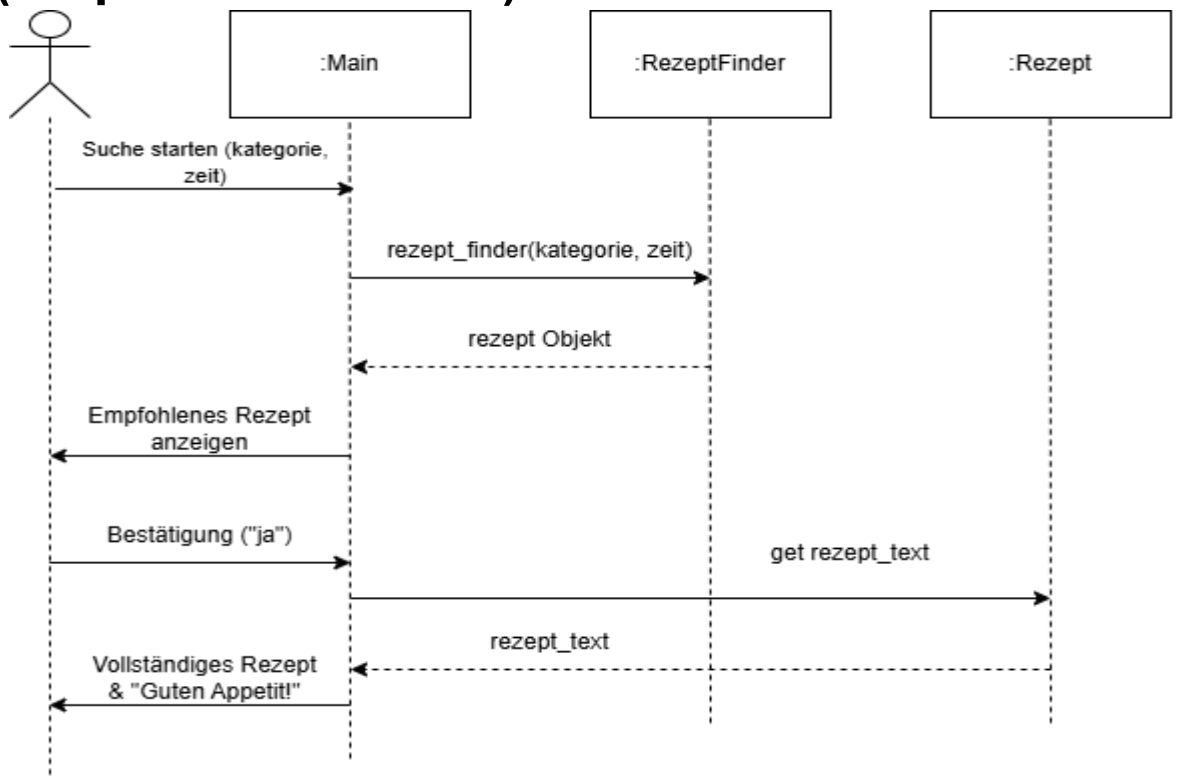
3. **Präsentationsschicht (View / GUI):** Die Klasse RezeptApp baut das TkinterFenster auf, nimmt die Eingaben des Benutzers entgegen und gibt das Ergebnis aus.

4.1Technologieauswahl und Rationale

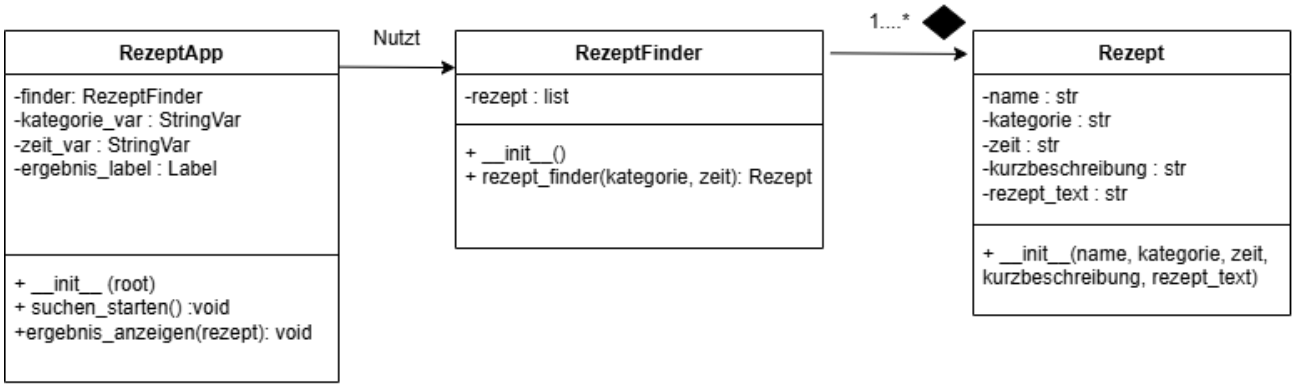
Für die Umsetzung meines Rezept-Empfehlungssystems habe ich mich für die Programmiersprache Python entschieden. Die Gründe dafür sind:

- Objektorientierte Programmierung (OOP): Python macht es leicht, Klassen und Objekte sauber zu trennen und zu strukturieren.
- Plattformunabhängigkeit: Das Programm läuft ohne Probleme auf verschiedenen Betriebssystemen (Windows, Mac, Linux).
- Einfache GUI mit Tkinter: Mit der eingebauten Bibliothek Tkinter konnte ich schnell und ohne komplizierte Installationen eine grafische Benutzeroberfläche mit Fenstern und Buttons bauen.

4.2 Architekturdokument: UML-Diagramme (Sequenz & Klassen)



UML-*Sequenzdiagramm*



UML-Klassendiagramm

Kurzbeschreibung zum Diagramm: Der Benutzer klickt in der Benutzeroberfläche auf die Kriterien und startet die Suche. Die App ruft den RezeptFinder auf, welcher ein passendes Rezept auswählt. Wenn der Benutzer auf "Gefällt mir" klickt, werden die Details aus dem Rezept-Objekt geladen und im Fenster angezeigt.

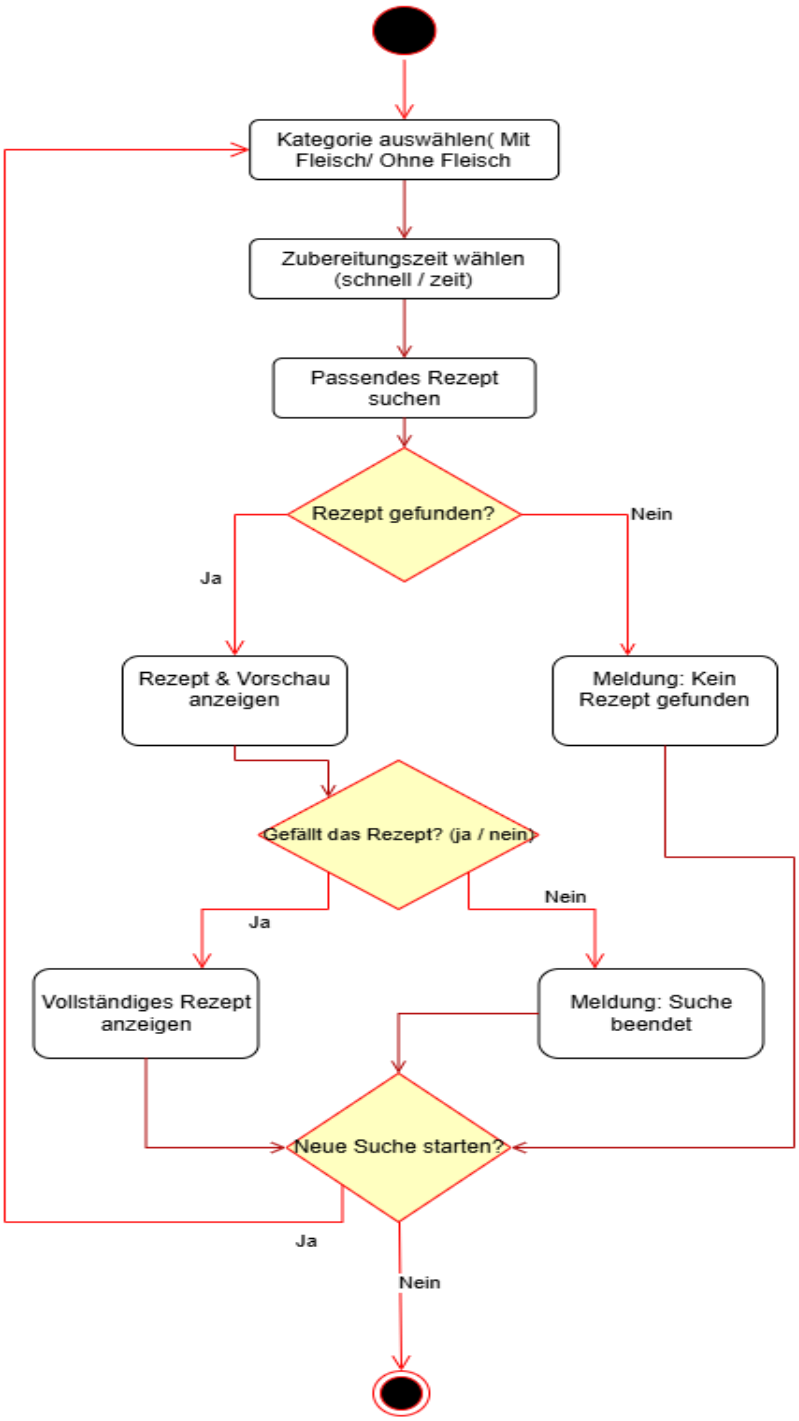
4.3Prozessabläufe (Aktivitätsdiagramm)

Stellt die objektorientierte Struktur des Programms dar. Die RezeptApp steuert die Benutzeroberfläche, der RezeptFinder filtert die Gerichte, und die Klasse Rezept speichert die konkreten Daten der einzelnen Rezepte.

Aktivitätsdiagramm

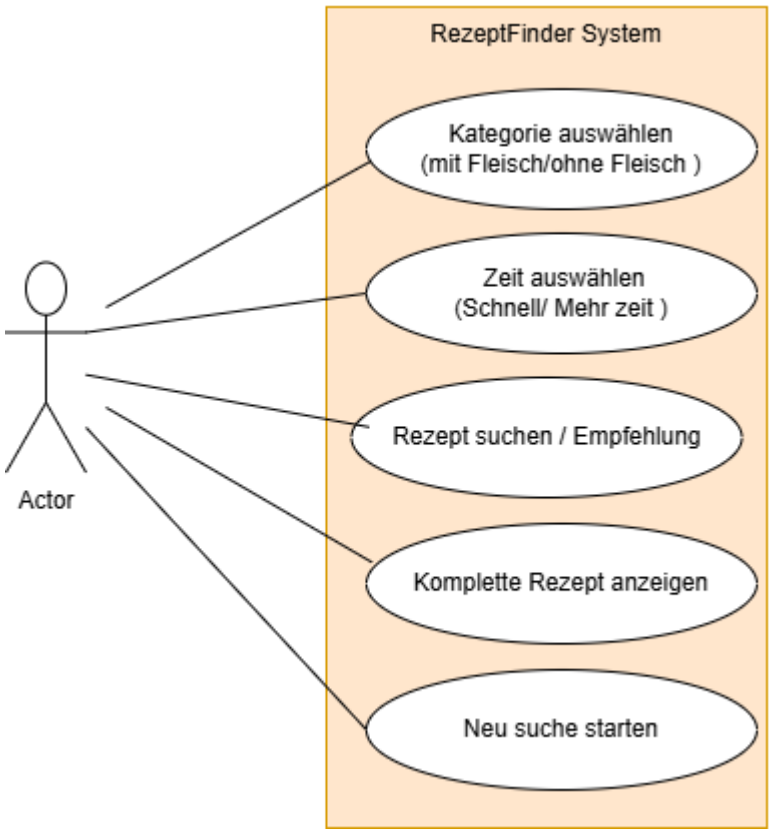
Das Aktivitätsdiagramm beschreibt den logischen Ablauf der Programmausführung:

- **Auswahl**: Der Benutzer wählt zunächst die gewünschte Kategorie (mit Fleisch / vegetarisch) und die Zubereitungszeit aus.
- **Suche & Prüfung**: Das System startet die Rezeptsuche. Wenn ein passendes Gericht gefunden wurde, wird eine Vorschau angezeigt. Falls kein Treffer erzielt wird, erhält der Benutzer eine entsprechende Meldung.
- **Ergebnis & Wiederholung**: Nach der Anzeige des Rezeptdetails kann der Benutzer entscheiden, ob das Gericht gefällt oder ob er eine neue Suche starten möchte.



UML-Use-Case-Diagramm

Zeigt den typischen Ablauf aus Sicht des Benutzers: Er wählt die Kategorie und die Zeit aus, startet die Suche, lässt sich das Rezept anzeigen oder beginnt auf Wunsch eine neue Suche.


RezeptFinder System
Kategorie auswählen
(mit Fleisch/ohne Fleisch)
Zeit auswählen
(Schnell/ Mehr zeit)
Rezept suchen / Empfehlung
Komplette Rezept anzeigen
Neu suche starten
Actor